Ligero 프로토콜 이론적 배경

개요

Ligero는 2017년 Ames, Hazay, Ishai, Venkitasubramaniam이 제안한 zkSNARK 프로토콜로, **O(√N) 크기의 증명**을 달성하는 것이 특징입니다. 이는 기존의 선형 크기 증명 시스템에 비해 상당한 개선을 나타내며, 특히 **직접 저차 테스트(Direct Low-Degree Test)**를 사용하여 구현의 단순성과 효율성을 동시에 달성합니다.

핵심 아이디어와 O(N^0.5) 달성 원리

1. 행렬 기반 인코딩 접근법

Ligero의 핵심 혁신은 R1CS 제약 조건을 **2차원 행렬 구조**로 재구성하는 것입니다:

```
증인 벡터 w를 m×n 행렬 W로 재배열

- m: 행렬의 높이 (height)

- n: 행렬의 너비 (width)

- m × n = |w| (증인 벡터의 크기)
```

이러한 행렬 구조를 통해 Ligero는 다음과 같은 이점을 얻습니다:

1. **행별 검증**: 각 행이 올바른 R1CS 제약을 만족하는지 독립적으로 검증
2. **열별 일관성**: 각 열이 Reed-Solomon 코드워드인지 확인
3. **쿼리 복잡도 감소**: √N개의 위치만 쿼리하여 전체 증명 검증

2. 높이-너비 비율 최적화

```
// libiop 구현에서의 매개변수 설정
float height_width_ratio_; // 기본값: 0.1
```

높이-너비 비율은 성능에 직접적인 영향을 미칩니다:

- **낮은 비율 (height < width)**:
- 장점: 적은 쿼리 수, 빠른 검증
- 단점: 더 많은 Reed-Solomon 인코딩 필요
- **높은 비율 (height > width)**:
- 장점: 적은 인코딩 오버헤드
- 단점: 더 많은 쿼리 필요

3. O(√N) 복잡도 달성 메커니즘

Ligero가 O(√N) 증명 크기를 달성하는 수학적 원리:

1. **행렬 차원**: m × n = N이고, 최적화를 위해 m ≈ √N, n ≈ √N
2. **쿼리 수**: 보안 매개변수 λ에 대해 O(λ) 개의 행과 열을 쿼리
3. **각 쿼리 크기**: 각 행/열은 O(√N) 크기
4. **총 증명 크기**: O(λ) × O(√N) = O(√N) (λ는 상수)

직접 LDT(Low-Degree Test)의 동작 방식

1. 직접 LDT의 기본 원리

기존의 FRI나 다른 재귀적 LDT와 달리, Ligero는 **직접적인 방법**으로 저차성을 검증합니다:

```
// direct_ldt.hpp에서의 핵심 구조
```

```
template<typename FieldT>
class direct_LDT_protocol {
// 증명자가 다항식의 계수를 직접 전송
std::vector<prover_message_handle> prover_coefficients_handles_;
// 검증자가 무작위 위치에서 쿼리
std::vector<random_query_position_handle> query_position_handles_;
};
```

2. 직접 LDT의 단계별 과정

단계 1: 계수 전송

- 증명자는 저차 다항식 P(x)의 **모든 계수**를 검증자에게 전송
- 다항식 차수가 d라면, d+1개의 계수 전송

단계 2: 일관성 검증

- 검증자는 k개의 무작위 위치 $\{x_1, x_2, ..., x_k\}$를 선택
- 각 위치에서 오라클 값 $P(x_i)$를 쿼리
- 전송받은 계수로 계산한 $P(x_i)$와 비교

단계 3: 사운드니스 분석

```
잘못된 다항식이 검증을 통과할 확률:
- 차수 d인 다항식이 k개 점에서 일치할 확률 ≤ d/|F|
- 보안 매개변수 λ에 대해 k = O(λ) 선택
```

```
- 사운드니스 오류: ≤ d/|F|^k = negligible
```

3. 직접 LDT의 장단점

장점:

1. **구현 단순성**: 복잡한 재귀 구조 불필요
2. **투명성**: 신뢰할 수 있는 설정 불필요
3. **병렬화**: 각 쿼리를 독립적으로 처리 가능
4. **메모리 효율성**: 중간 다항식 저장 불필요

단점:

1. **증명 크기**: O(√N)로 FRI의 O(log N)보다 큼
2. **계수 전송**: 전체 다항식 계수 전송 필요
3. **쿼리 수**: 높은 보안을 위해 많은 쿼리 필요

2라운드 프로토콜 구조와 상호작용 패턴

1. 라운드 구조 개요

Ligero는 **2라운드 공개 코인 프로토콜**로 설계되었습니다:

```
라운드 1 (증명자 → 검증자):
- 인코딩된 증인 행렬 전송
- Reed-Solomon 코드워드들의 커밋먼트
- 블라인딩 벡터 (영지식성을 위해)
```

```
라운드 2 (검증자 → 증명자):

- 무작위 쿼리 위치 선택

- 행과 열의 무작위 선형 결합 계수



라운드 3 (증명자 → 검증자):

- 쿼리된 위치의 값들

- 머클 트리 인증 경로

- 선형 결합 결과
```

2. 상호작용 패턴 상세 분석

2.1 첫 번째 라운드: 커밋먼트 단계

```
// ligero.hpp에서의 오라클 핸들 구조

std::vector<oracle_handle_ptr> w_vector_handles_; // 증인 벡터

std::vector<oracle_handle_ptr> a_vector_handles_; // A 행렬 인코딩

std::vector<oracle_handle_ptr> b_vector_handles_; // B 행렬 인코딩

std::vector<oracle_handle_ptr> c_vector_handles_; // C 행렬 인코딩
```

증명자의 작업:

1. **행렬 구성**: 증인 w를 m×n 행렬로 재배열
2. **Reed-Solomon 인코딩**: 각 행을 RS 코드워드로 확장
3. **커밋먼트**: 머클 트리를 사용하여 모든 값에 커밋

2.2 두 번째 라운드: 도전 단계

검증자의 작업:

1. **행 선택**: 무작위로 r개의 행 인덱스 선택
2. **열 선택**: 무작위로 c개의 열 인덱스 선택
3. **선형 결합**: 무작위 계수로 선형 결합 구성

2.3 세 번째 라운드: 응답 단계

증명자의 작업:

1. **값 공개**: 선택된 위치의 모든 값 공개
2. **인증 경로**: 머클 트리 인증 경로 제공
3. **일관성 증명**: 행과 열의 일관성 증명

3. 영지식성 구현

```
// 영지식성을 위한 블라인딩 벡터
std::vector<oracle_handle_ptr> lincheck_A_blinding_vector_handles_;
std::vector<oracle_handle_ptr> lincheck_B_blinding_vector_handles_;
std::vector<oracle_handle_ptr> lincheck_C_blinding_vector_handles_;
std::vector<oracle_handle_ptr> rowcheck_blinding_vector_handles_;
```

영지식성 달성 방법:

1. **블라인딩 다항식**: 각 인코딩에 무작위 블라인딩 다항식 추가
2. **제로섬 조건**: 블라인딩 벡터들의 합이 0이 되도록 구성
3. **마스킹**: 실제 증인 정보를 무작위성으로 마스킹

4. 보안성 분석

4.1 완전성 (Completeness)

- 정직한 증명자는 항상 검증을 통과
- 올바른 R1CS 만족 할당에 대해 모든 검사 통과

4.2 사운드니스 (Soundness)

- 잘못된 증명자가 검증을 통과할 확률은 negligible
- 각 라운드에서의 오류 확률이 곱해져 전체 오류 감소

4.3 영지식성 (Zero-Knowledge)

- 시뮬레이터가 실제 증인 없이 동일한 분포의 증명 생성 가능
- 블라인딩 벡터를 통한 정보 은닉

성능 특성 및 최적화

1. 복잡도 분석

| 메트릭 | Ligero | 비고 |

|--------|--------|------|

| 증명 크기 | O(√N) | N은 회로 크기 |

| 증명 시간 | O(N log N) | FFT 연산 포함 |

| 검증 시간 | O(√N) | 쿼리 수에 비례 |

| 라운드 수 | 2 | 공개 코인 |

2. 실용적 고려사항

2.1 메모리 사용량

```
// 행렬 차원 설정이 메모리에 미치는 영향
std::size_t matrix_height_; // m
std::size_t matrix_width_; // n
// 총 메모리: O(m × n × expansion_factor)
```

2.2 병렬화 가능성

- **행별 처리**: 각 행의 RS 인코딩을 병렬로 수행
- **쿼리 처리**: 독립적인 쿼리들을 병렬로 검증
- **FFT 연산**: 벡터화된 FFT 구현 활용

2.3 하드웨어 최적화

- **SIMD 명령어**: 벡터 연산에 SIMD 활용
- **캐시 효율성**: 행렬 접근 패턴 최적화
- **메모리 대역폭**: 스트리밍 접근으로 대역폭 활용

이러한 이론적 배경을 바탕으로 Ligero는 실용적인 zkSNARK 시스템으로서 투명성과 효율성의 균형을 달성하며, 특히 중간 규모의 회로에서 우수한 성능을 보입니다.